

Whenever there's enough horizontal space to continue the loop, we want to do two things: create an alien at the correct position, and define the horizontal position of the next alien in the row. We create an alien and assign it to `new_alien` ❸. Then we set the precise horizontal position to the current value of `current_x` ❹. We also position the alien's rect at this same `x`-value, and add the new alien to the group `self.aliens`.

Finally, we increment the value of `current_x` ❺. We add two alien widths to the horizontal position, to move past the alien we just added and to leave some space between the aliens as well. Python will re-evaluate the condition at the start of the while loop and decide if there's room for another alien. When there's no room left, the loop will end, and we should have a full row of aliens.

When you run *Alien Invasion* now, you should see the first row of aliens appear, as in Figure 13-3.

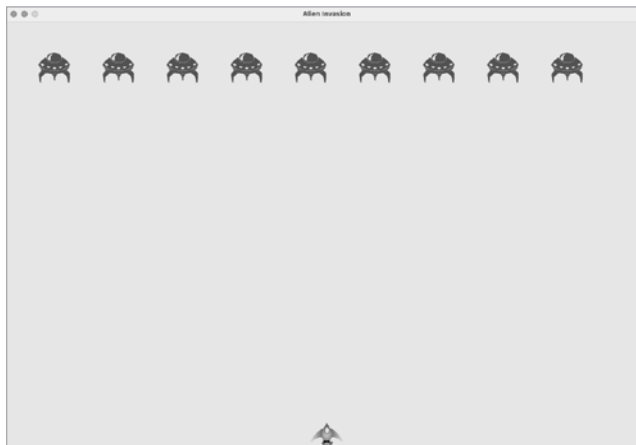


Figure 13-3: The first row of aliens

**NOTE**

*It's not always obvious exactly how to construct a loop like the one shown in this section. One nice thing about programming is that your initial ideas for how to approach a problem like this don't have to be correct. It's perfectly reasonable to start a loop like this with the aliens positioned too far to the right, and then modify the loop until you have an appropriate amount of space on the screen.*

## Refactoring `_create_fleet()`

If the code we've written so far was all we needed to create a fleet, we'd probably leave `_create_fleet()` as is. But we have more work to do, so let's clean up the method a bit. We'll add a new helper method, `_create_alien()`, and call it from `_create_fleet()`:

`alien`  
`_invasion.py`

```
def _create_fleet(self):
    --snip--
    while current_x < (self.settings.screen_width - 2 * alien_width):
        self._create_alien(current_x)
        current_x += 2 * alien_width
```